

EMSI Casablanca

Ecole Marocaine des Sciences de l'Ingenieur

Filiere Informatique | Annee 2025-2026

Fiche de synthese structuree

Module : Deep Learning

MLP et PyTorch • CNN et Vision • RNN / LSTM / GRU / Seq2Seq

Datasets utilises :

Breast Cancer Wisconsin | MNIST | Corpus Sentiment & Traduction FR-EN

Travail individuel | Annee universitaire 2025-2026

Table des matieres

1. Vue d'ensemble	4
2. Definitions essentielles.....	4
2.1 nn.Module	4
2.2 Parametre	4
2.3 Gradient	4
2.4 state_dict.....	4
2.5 Device	4
3. Construction d'un MLP en PyTorch.....	4
3.1 Avec nn.Sequential.....	4
3.2 Avec une classe personnalisee	5
4. Formules fondamentales.....	5
5. Gestion des parametres.....	5
5.1 Inspection avec named_parameters().....	5
5.2 Acces au state_dict	6
6. Initialisation des parametres.....	6
6.1 Initialisation Gaussienne.....	6
6.2 Initialisation Constante	6
6.3 Initialisation Xavier (recommandee).....	6
7. Sauvegarde et rechargement.....	7
8. Utilisation du GPU.....	7
9. Resultats sur Breast Cancer Wisconsin	7
10. Erreurs frequentes	7
11. Question de synthese — Partie I.....	7
12. Resume final — Partie I	8
13. Vue d'ensemble	9
14. Definitions essentielles.....	9
14.1 Correlation croisee 2D	9
14.2 Padding	9
14.3 Stride.....	9
14.4 Pooling	9
14.5 Convolution 1x1.....	9
15. Formules dimensionnelles.....	9
16. Implementation manuelle	10
16.1 Correlation croisee 2D	10
16.2 Max-Pooling et Average-Pooling manuels	10
17. Architecture CNN (LeNet)	10
18. Resultats experimentaux.....	11
18.1 Comparaison MLP vs CNN sur MNIST	11

18.2 Influence des hyperparametres	11
19. Visualisation des feature maps	11
20. Erreurs frequentes	11
21. Question de synthese — Partie II.....	12
22. Resume final — Partie II	12
23. Vue d'ensemble	13
24. Definitions essentielles.....	13
24.1 Hidden State.....	13
24.2 Perplexite	13
24.3 BPTT	13
24.4 Gradient Clipping.....	13
24.5 Teacher Forcing	13
25. Formules fondamentales.....	13
26. Implementation RNN, LSTM, GRU.....	14
26.1 RNN Simple.....	14
26.2 LSTM.....	14
26.3 GRU	14
27. Comparaison RNN / LSTM / GRU.....	14
28. Gradient Clipping et BPTT	15
29. Systeme Seq2Seq	15
29.1 Architecture encodeur-decodeur.....	15
30. Strategies de decodage	15
31. Erreurs frequentes	16
32. Question de synthese — Partie III.....	16
33. Resume final — Partie III	16
34. Synthese comparative des architectures.....	17
35. Liens conceptuels entre les architectures.....	17
36. Conclusion generale	17

PARTIE I — MLP ET PYTORCH

Construction des modeles • Gestion des parametres • Initialisation • Sauvegarde • GPU

Cette partie presente les concepts fondamentaux de PyTorch pour la construction de perceptrons multicouches (MLP), avec definitions, formules, scripts commentes et analyse des resultats experimentaux obtenus sur le dataset Breast Cancer Wisconsin.

1. Vue d'ensemble

En PyTorch, la brique fondamentale de modelisation est la classe `nn.Module`. Un module peut représenter une couche, un bloc de calcul ou un reseau complet. Il recoit une entree, calcule une sortie via la methode `forward`, et peut contenir des parametres apprenables.

- Construire un modele avec `nn.Sequential` ou une classe personnalisée.
- Inspecter et gerer les parametres avec `named_parameters()` et `state_dict()`.
- Initialiser correctement les poids pour stabiliser l'apprentissage.
- Sauvegarder et recharger les modeles avec `torch.save` et `torch.load`.
- Utiliser le GPU en gardant modele et donnees sur le meme device.

2. Definitions essentielles

2.1 `nn.Module`

Un module est un objet derive de `nn.Module`. Il encapsule une transformation de l'entree vers la sortie et s'integre automatiquement au calcul des gradients via `autograd`.

2.2 Parametre

Un parametre est une variable apprenable du modele. En pratique, il s'agit des poids (weight) et des biais (bias) des couches lineaires.

2.3 Gradient

Le gradient mesure la sensibilite de la fonction de perte par rapport a un parametre. Il est calcule par `loss.backward()` et utilise par l'optimiseur pour mettre a jour les poids.

2.4 `state_dict`

Le `state_dict()` est un dictionnaire ordonne qui associe a chaque nom de parametre son tenseur. C'est le format standard de sauvegarde et de rechargement des parametres.

2.5 Device

Un device indique l'emplacement memoire d'un tenseur ou d'un modele : CPU ou GPU (cuda). Toutes les donnees impliquees dans une meme operation doivent etre sur le meme device.

3. Construction d'un MLP en PyTorch

3.1 Avec `nn.Sequential`

`nn.Sequential` enchaîne les modules dans l'ordre. C'est la solution la plus rapide pour des architectures lineaires simples.

```
import torch
from torch import nn

# VERSION 1 : nn.Sequential
mlp_sequential = nn.Sequential(
    nn.Linear(30, 64),      # 30 features -> 64 neurones caches
    nn.ReLU(),             # Activation non-lineaire
    nn.Dropout(0.3),       # Regularisation
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(32, 2)       # 2 classes : malin / benin
)
```

3.2 Avec une classe personnalisée

Cette approche est plus flexible : elle permet des architectures non lineaires, des branchements et des opérations spécifiques dans `forward()`.

```
class MLP(nn.Module):
    def __init__(self, input_dim=30, hidden1=64, hidden2=32, output_dim=2):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden1)
        self.relu1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.relu2 = nn.ReLU()
        self.drop2 = nn.Dropout(0.3)
        self.fc3 = nn.Linear(hidden2, output_dim)

    def forward(self, x):
        x = self.drop1(self.relu1(self.fc1(x)))
        x = self.drop2(self.relu2(self.fc2(x)))
        return self.fc3(x)

mlp_custom = MLP()    # 4130 parametres au total
```

4. Formules fondamentales

Concept	Formule / Explication
Couche lineaire	$y = x * W^T + b$
Couche cachee MLP	$H = \text{ReLU}(x * W1 + b1)$
Sortie MLP	$Y = H * W2 + b2$
Activation ReLU	$\text{ReLU}(z) = \max(0, z)$
Dropout	Annule aleatoirement p% des neurones a l'entrainement
Cross-Entropy Loss	$L = -\sum(y_i * \log(p_i))$
Mise a jour Adam	$\theta = \theta - lr * m / (\sqrt{v} + \epsilon)$

5. Gestion des parametres

5.1 Inspection avec `named_parameters()`

```
total = 0
for name, param in mlp_custom.named_parameters():
    print(f'{name:20s} | shape: {str(param.shape):25s} | params: {param.numel()}')
```

```

    total += param.numel()
print(f'Total parametres : {total}')
```

Resultat obtenu :

```

# fc1.weight | shape: torch.Size([64, 30]) | params: 1920
# fc1.bias   | shape: torch.Size([64])      | params: 64
# fc2.weight | shape: torch.Size([32, 64])  | params: 2048
# fc3.weight | shape: torch.Size([2, 32])     | params: 64
# Total parametres : 4130
```

5.2 Acces au state_dict

```

for key in mlp_custom.state_dict().keys():
    print(key)
# fc1.weight | fc1.bias | fc2.weight | fc2.bias | fc3.weight | fc3.bias

print(mlp_custom.fc1.weight.data)    # valeurs numeriques
print(mlp_custom.fc1.weight.grad)    # gradient (None avant backward())
```

6. Initialisation des parametres

6.1 Initialisation Gaussienne

```

def init_gaussienne(model):
    for m in model.modules():
        if isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, mean=0, std=0.01)
            nn.init.zeros_(m.bias)
```

Garde les poids petits au depart. Peut causer des gradients qui disparaissent sur les reseaux profonds.

6.2 Initialisation Constante

```

def init_constant(model):
    for m in model.modules():
        if isinstance(m, nn.Linear):
            nn.init.constant_(m.weight, 0.01)
            nn.init.zeros_(m.bias)
```

Problematique en pratique : tous les neurones commencent identiques. C'est le probleme de symetrie — ils apprennent exactement la meme chose.

6.3 Initialisation Xavier (recommandee)

```

def init_xavier(model):
    for m in model.modules():
        if isinstance(m, nn.Linear):
            nn.init.xavier_uniform_(m.weight)
            nn.init.zeros_(m.bias)
```

Xavier stabilise la variance des signaux lors de la propagation. C'est la strategie la plus adaptee aux reseaux profonds avec ReLU.

Initialisation	Val Acc	Val Loss	Observation
Gaussienne	100%	0.0009	Convergence rapide
Constante	94.19%	0.2093	Lente, probleme symetrie
Xavier	100%	0.0005	Meilleure — recommandee

7. Sauvegarde et rechargement

```
# Sauvegarder le meilleur modele
torch.save({
    'model_state_dict' : model_xavier.state_dict(),
    'architecture'      : 'MLP(30, 64, 32, 2)',
    'initialisation'     : 'Xavier',
}, 'models/meilleur_modele.pth')

# Recharger le modele
model_charge = MLP().to(device)
checkpoint    = torch.load('models/meilleur_modele.pth', map_location=device)
model_charge.load_state_dict(checkpoint['model_state_dict'])
model_charge.eval()      # IMPORTANT : desactive Dropout pour l'inference
```

8. Utilisation du GPU

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model   = MLP().to(device)      # modele sur le device
X_data  = X_data.to(device)     # donnees sur le MEME device
y_data  = y_data.to(device)

# REGLE ABSOLUE : modele et donnees doivent etre sur le MEME device
# Sinon PyTorch leve une RuntimeError
```

9. Resultats sur Breast Cancer Wisconsin

Metrique	Valeur	Interpretation
Accuracy	94.19%	94% des predictions sont correctes
Precision	98.04%	Peu de faux positifs
Recall	92.59%	Detecte 92.6% des tumeurs malignes
F1-Score	95.24%	Bon equilibre precision / rappel

10. Erreurs frequentes

- Oublier `super().__init__()` dans une classe heritee de `nn.Module`.
- Ne pas appeler `.eval()` avant l'inference : le Dropout reste actif.
- Melanger CPU et GPU dans une meme operation : `RuntimeError` inevitable.
- Appliquer `fit_transform` sur le test set : fuite de donnees.
- Oublier `optimizer.zero_grad()` avant `loss.backward()` : accumulation des gradients.
- Initialisation constante : probleme de symetrie, tous les neurones apprennent la meme chose.

11. Question de synthese — Partie I

Dans quelle mesure un MLP bien parametre constitue-t-il une solution pertinente pour la classification tabulaire ?

Le MLP est une solution pertinente et efficace pour les donnees tabulaires structurees. Sur Breast Cancer Wisconsin (569 echantillons, 30 features), il atteint 94.19% d'accuracy avec seulement 4130 parametres grace a l'initialisation Xavier, la normalisation StandardScaler et la regularisation Dropout(0.3).

Ses limites apparaissent sur des donnees avec une structure particuliere : il ne peut pas exploiter la structure spatiale des images (CNN preferable) ni la dependance temporelle des sequences (RNN preferable). Il est aussi sensible au desequilibre des classes (212 malins vs 357 benins) et necessite une normalisation obligatoire.

12. Resume final — Partie I

`nn.Module` est l'abstraction centrale de PyTorch. Les parametres doivent etre bien initialises (Xavier de preference), inspectes via `named_parameters()` et sauvegardes via `state_dict()`. Le GPU apporte un gain de vitesse important, a condition que donnees et modele soient sur le meme device. Le MLP reste la reference pour les donnees tabulaires sans structure spatiale ou temporelle.

PARTIE II — CNN ET VISION PAR ORDINATEUR

Convolution • Pooling • LeNet • Comparaison MLP vs CNN • Feature Maps

Cette partie presente les concepts fondamentaux des reseaux de neurones convolutionnels (CNN) pour la classification d'images, avec definitions, calculs dimensionnels, scripts commentes et resultats experimentaux sur MNIST.

13. Vue d'ensemble

Le CNN (Convolutional Neural Network) est l'architecture de reference pour les images. Contrairement au MLP qui aplati l'image et perd sa structure spatiale, le CNN exploite trois proprietes fondamentales :

- Localite : chaque filtre detecte un motif local (bord, courbe, texture) dans une region limitee.
- Partage des poids : le meme filtre est applique sur toute l'image, reduisant le nombre de parametres.
- Hierarchie : premieres couches = bords, couches profondes = formes complexes.

14. Definitions essentielles

14.1 Correlation croisee 2D

L'operation fondamentale du CNN. Un filtre (kernel) K est glisse sur l'image X pour produire une carte de caracteristiques (feature map).

14.2 Padding

Ajout de zeros autour de l'image avant la convolution. Permet de controler la taille de sortie et de preserver l'information sur les bords.

14.3 Stride

Pas de deplacement du filtre. Un stride de 2 divise la taille de sortie par ~ 2 et accelere le calcul.

14.4 Pooling

Reduction de la dimensionnalite spatiale. Max Pooling = valeur maximale, Average Pooling = moyenne dans chaque fenetre.

14.5 Convolution 1x1

Kernel de taille 1x1. Modifie le nombre de canaux sans affecter la taille spatiale. Utile pour la reduction de dimensionnalite.

15. Formules dimensionnelles

Operation	Formule
Taille sortie convolution	$(\text{Input} - \text{Kernel} + 2 \times \text{Padding}) / \text{Stride} + 1$
Taille sortie pooling	$(\text{Input} - \text{Pool}) / \text{Stride} + 1$
Params couche conv	$\text{Kernel_H} \times \text{Kernel_W} \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$

Conv1 LeNet (28x28)	$(28 - 5 + 2 \times 2) / 1 + 1 = 28 \times 28$
Pool1 LeNet	$(28 - 2) / 2 + 1 = 14 \times 14$
Conv2 LeNet (14x14)	$(14 - 5 + 0) / 1 + 1 = 10 \times 10$
Pool2 LeNet	$(10 - 2) / 2 + 1 = 5 \times 5$

16. Implementation manuelle

16.1 Correlation croisee 2D

```
def correlation_croisee_2d(X, K):
    h_x, w_x = X.shape
    h_k, w_k = K.shape
    h_out = h_x - h_k + 1
    w_out = w_x - w_k + 1
    Y = torch.zeros(h_out, w_out)
    for i in range(h_out):
        for j in range(w_out):
            Y[i, j] = (X[i:i+h_k, j:j+w_k] * K).sum()
    return Y

X = torch.tensor([[1., 2., 3.], [4., 5., 6.], [7., 8., 9.]])
K = torch.tensor([[1., 0.], [0., 1.]])
print(correlation_croisee_2d(X, K)) # tensor([[6., 8.], [12., 14.]])
```

16.2 Max-Pooling et Average-Pooling manuels

```
def max_pooling_2d(X, pool_size=2, stride=2):
    h, w = X.shape
    h_out = (h - pool_size) // stride + 1
    w_out = (w - pool_size) // stride + 1
    Y = torch.zeros(h_out, w_out)
    for i in range(h_out):
        for j in range(w_out):
            Y[i, j] = X[i*stride:i*stride+pool_size,
                        j*stride:j*stride+pool_size].max()
    return Y

# Verification avec PyTorch
X_4d = X_pool.unsqueeze(0).unsqueeze(0)
mp_torch = F.max_pool2d(X_4d, kernel_size=2, stride=2).squeeze()
print('Identique PyTorch :', torch.allclose(max_pooling_2d(X_pool), mp_torch))
```

17. Architecture CNN (LeNet)

```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, padding=2) # 28->28
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2) # 28->14
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5) # 14->10
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2) # 10->5
        self.conv1x1 = nn.Conv2d(16, 16, kernel_size=1) # 5->5
        self.fc1 = nn.Linear(16*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool1(torch.relu(self.conv1(x)))
```

```
x = self.pool2(torch.relu(self.conv2(x)))
x = torch.relu(self.conv1x1(x))
x = x.view(x.size(0), -1) # aplatissement
x = torch.relu(self.fc1(x))
x = torch.relu(self.fc2(x))
return self.fc3(x)
```

18. Resultats experimentaux

18.1 Comparaison MLP vs CNN sur MNIST

Modele	Test Accuracy	Parametres	Observation
MLP (784->256->128->10)	~97%	201,994	Perd la structure spatiale
CNN LeNet	~99%	61,978	Exploite la localite spatiale

18.2 Influence des hyperparametres

Configuration	Test Accuracy	Observation
Sans padding (kernel=3)	~97%	Perte info sur les bords
Avec padding=1 (kernel=3)	~98%	Preserve la taille spatiale
Plus de filtres (32 et 64)	~99%	Plus de motifs detectes

19. Visualisation des feature maps

Les feature maps montrent ce que chaque filtre detecte. Les filtres de conv1 (6 filtres) detectent des structures de bas niveau (bords, contrastes). Les filtres de conv2 (16 filtres) detectent des formes plus complexes (courbes, jonctions).

```
activation = {}
def hook_fn(module, input, output):
    activation['output'] = output.detach()

handle = model_cnn.conv1.register_forward_hook(hook_fn)
with torch.no_grad():
    _ = model_cnn(img_tensor)
handle.remove()

fmaps = activation['output'].squeeze()
for i in range(6): # visualiser les 6 filtres de conv1
    plt.subplot(2, 3, i+1)
    plt.imshow(fmaps[i].cpu(), cmap='viridis')
```

20. Erreurs frequentes

- Oublier d'aplatir le tenseur avant les FC : `x = x.view(x.size(0), -1)`.
- Confondre `padding='same'` et `padding=0` : la taille de sortie change radicalement.
- Ne pas tenir compte des canaux dans le calcul des parametres.
- Oublier que le nb de canaux d'entree de conv2 = nb de filtres de conv1.

- Utiliser un stride trop grand qui perd trop d'information spatiale.

21. Question de synthese — Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images ?

Le CNN est superieur au MLP pour les images car il exploite la structure spatiale 2D. En aplatissant l'image, le MLP traite chaque pixel independamment et perd toute notion de voisinage. Le CNN detecte des motifs locaux (bords, textures, formes) et les compose hierarchiquement pour former des representations abstraites.

Le partage des poids est l'avantage majeur : un meme filtre est applique sur toute l'image, reduisant le nombre de parametres (61,978 vs 201,994) tout en ameliorant les performances (99% vs 97%). Le padding preserve l'info sur les bords, le pooling apporte l'invariance aux translations.

22. Resume final — Partie II

Le CNN est l'architecture de reference pour les images. Il exploite la localite spatiale, le partage des poids et la hierarchie des representations. L'architecture LeNet atteint 99% sur MNIST avec 3x moins de parametres que le MLP. Les hyperparametres cles sont le padding, le nombre de filtres et la profondeur.

PARTIE III — RNN, LSTM, GRU ET SEQ2SEQ

Sequences • BPTT • Gradient Clipping • Encodeur-Decodeur • Beam Search

Cette partie presente les architectures recurrentes pour la modelisation de sequences, avec definitions, formules, scripts commentes, systeme Seq2Seq et resultats experimentaux.

23. Vue d'ensemble

Les donnees sequentielles ont une structure temporelle que le MLP et le CNN ne peuvent pas exploiter. Les architectures recurrentes maintiennent un etat cache qui encode le contexte accumule.

- RNN simple : traite les sequences mais souffre du gradient qui disparaît.
- LSTM : resout le probleme grace a une cellule memoire et 3 portes.
- GRU : version simplifiee du LSTM avec 2 portes, plus rapide.
- Seq2Seq : encodeur-decodeur pour les taches de generation/traduction.

24. Definitions essentielles

24.1 Hidden State

Le hidden state h_t est la memoire du reseau a l'instant t . Il encode le contexte de tous les tokens precedents et est passe comme entree au pas de temps suivant.

24.2 Perplexite

Mesure la qualite d'un modele de langage. $PPL = \exp(\text{CrossEntropyLoss})$. Plus elle est basse, meilleur est le modele. Une perplexite proche de 1 = modele quasi-parfait.

24.3 BPTT

La retropropagation a travers le temps calcule les gradients en deroulant le reseau sur toute la sequence. Sur les longues sequences, les gradients peuvent disparaître ou exploser.

24.4 Gradient Clipping

Limite la norme des gradients a une valeur maximale (max_norm) pour eviter l'explosion du gradient et stabiliser l'entrainement.

24.5 Teacher Forcing

Donne le vrai token precedent au decodeur avec une probabilite p pendant l'entrainement. Accelere la convergence mais peut creer un ecart entrainement/inference.

25. Formules fondamentales

Concept	Formule
Modele de langage	$P(w_1, \dots, w_n) = P(w_1) \times P(w_2 w_1) \times \dots \times P(w_n w_1, \dots, w_{n-1})$
RNN hidden state	$h_t = \tanh(W_h * h_{t-1} + W_x * x_t + b)$
LSTM porte oubli	$f_t = \text{sigma}(W_f * [h_{t-1}, x_t] + b_f)$

LSTM porte entree	$i_t = \text{sigma}(W_i * [h_{t-1}, x_t] + b_i)$
LSTM porte sortie	$o_t = \text{sigma}(W_o * [h_{t-1}, x_t] + b_o)$
LSTM cellule memoire	$c_t = f_t * c_{t-1} + i_t * \tanh(g_t)$
LSTM hidden state	$h_t = o_t * \tanh(c_t)$
Perplexite	$\text{PPL} = \exp(\text{CrossEntropyLoss})$
Gradient clipping	$g = g * (\text{max_norm} / \ g\)$ si $\ g\ > \text{max_norm}$

26. Implementation RNN, LSTM, GRU

26.1 RNN Simple

```
class RNN_Simple(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, EMBED_DIM, padding_idx=0)
        self.rnn = nn.RNN(EMBED_DIM, HIDDEN_DIM, batch_first=True)
        self.fc = nn.Linear(HIDDEN_DIM, OUTPUT_DIM)

    def forward(self, x):
        embedded = self.embedding(x)
        output, hidden = self.rnn(embedded)
        return self.fc(hidden[-1]) # dernier hidden state -> classification
```

26.2 LSTM

```
class LSTM_Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, EMBED_DIM, padding_idx=0)
        self.lstm = nn.LSTM(EMBED_DIM, HIDDEN_DIM, batch_first=True)
        self.fc = nn.Linear(HIDDEN_DIM, OUTPUT_DIM)

    def forward(self, x):
        embedded = self.embedding(x)
        out, (hn, cn) = self.lstm(embedded) # hn=hidden, cn=cellule memoire
        return self.fc(hn[-1])
```

26.3 GRU

```
class GRU_Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, EMBED_DIM, padding_idx=0)
        self.gru = nn.GRU(EMBED_DIM, HIDDEN_DIM, batch_first=True)
        self.fc = nn.Linear(HIDDEN_DIM, OUTPUT_DIM)

    def forward(self, x):
        embedded = self.embedding(x)
        out, hidden = self.gru(embedded) # GRU : pas de cellule memoire separee
        return self.fc(hidden[-1])
```

27. Comparaison RNN / LSTM / GRU

Critere	RNN Simple	LSTM	GRU
---------	------------	------	-----

Portes	Aucune	3 (oubli, entree, sortie)	2 (reset, update)
Parametres	2,658	7,458	5,858
Memoire long terme	Faible	Excellente	Bonne
Vitesse	Rapide	Lent	Moyen
Stabilite gradient	Faible	Elevee	Elevee
Usage recommande	Sequences courtes	Sequences longues	Bon compromis

28. Gradient Clipping et BPTT

```
for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()
    outputs = model(X_batch)
    loss = criterion(outputs, y_batch)
    loss.backward() # BPTT : retropropagation a travers le temps

    # Gradient clipping : evite l'explosion du gradient
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

    optimizer.step()
```

Sans gradient clipping, les gradients peuvent exploser sur les longues sequences. Le clipping maintient les gradients dans une plage raisonnable et stabilise l'entrainement.

29. Systeme Seq2Seq

29.1 Architecture encodeur-decodeur

Adapte aux taches ou entree et sortie ont des longueurs differentes (traduction). L'encodeur compresse la source en un vecteur de contexte, le decodeur genere la cible mot par mot.

```
class Seq2Seq(nn.Module):
    def __init__(self, encodeur, decodeur):
        super().__init__()
        self.encodeur = encodeur
        self.decodeur = decodeur

    def forward(self, src, tgt, teacher_forcing=0.5):
        _, hidden = self.encodeur(src) # compression de la source
        input_token = tgt[:, 0] # token <SOS>
        outputs = []
        for t in range(1, tgt.shape[1]):
            pred, hidden = self.decodeur(input_token, hidden)
            outputs.append(pred)
            use_tf = random.random() < teacher_forcing
            input_token = tgt[:, t] if use_tf else pred.argmax(1)
        return torch.stack(outputs, dim=1)
```

30. Strategies de decodage

Strategie	Principe / Avantage / Inconvenient
-----------	------------------------------------

Glouton	Choisit le mot le plus probable Rapide $O(n)$ Sous-optimal
Beam Search (k=3)	Explore k hypotheses Meilleure qualite Plus lent $O(k*n)$

31. Erreurs frequentes

- Oublier le gradient clipping : gradient qui explose sur les longues sequences.
- Confondre le hidden state du LSTM (h_n, c_n) et du GRU (h_n uniquement).
- Teacher forcing trop eleve : dependance aux vrais tokens, echec a l'inference.
- Oublier ignore_index=PAD_token dans CrossEntropyLoss.
- Ne pas reinitialiser le hidden state entre les batches independants.

32. Question de synthese — Partie III

Dans quelle mesure les architectures recurrentes permettent-elles de modeliser une sequence reelle ?

Le RNN simple est limite par le gradient qui disparaît sur les longues sequences. Le LSTM resout ce probleme avec une cellule memoire explicite (c_t) et trois portes selectant ce qui est retenu ou oublie. Le GRU offre un bon compromis avec 2 portes et moins de parametres.

Le Seq2Seq est justifie pour les taches ou entree et sortie ont des longueurs differentes. L'encodeur compresse la source, le decodeur genere la cible avec teacher forcing. Le beam search ameliore la qualite en explorant plusieurs hypotheses simultanement. La limite principale est le goulot d'etranglement du vecteur de contexte, que les Transformers resolvent avec l'attention.

33. Resume final — Partie III

Les architectures recurrentes sont adaptees aux sequences grace au hidden state qui encode le contexte temporel. Le LSTM et le GRU resolvent le probleme de gradient du RNN. Le Seq2Seq avec encodeur-decodeur traite des sequences de longueurs differentes. Le gradient clipping et le teacher forcing sont essentiels. Les Transformers (BERT, GPT) ont depasse les RNN grace au mecanisme d'attention globale.

QUESTION TRANSVERSALE FINALE

34. Synthèse comparative des architectures

Type de données	Architecture	Biais inductif	Mécanisme clé	Dataset
Tabulaire	MLP	Aucun	Fully connected	Breast Cancer (94.19%)
Image 2D	CNN	Localité spatiale	Filtres + pooling	MNIST (99%)
Sequence	RNN/LSTM/GRU	Ordre temporel	Hidden state + portes	Sentiment / FR-EN

35. Liens conceptuels entre les architectures

Malgré leurs différences architecturales, MLP, CNN et RNN partagent le même paradigme :

- Propagation avant (forward pass) : calcul des prédictions.
- Calcul de la loss (CrossEntropyLoss) : mesure de l'erreur.
- Retropropagation (backward pass) : calcul des gradients.
- Mise à jour des poids (Adam) : optimisation itérative.

Ce qui change selon la géométrie des données :

- MLP : aucune hypothèse sur la structure, traite chaque feature indépendamment.
- CNN : exploite la localité spatiale avec le partage des poids.
- RNN : exploite l'ordre temporel avec la récurrence.
- LSTM/GRU : résout le problème de gradient du RNN avec des portes.
- Transformer : remplace la récurrence par l'attention globale (BERT, GPT).

36. Conclusion générale

Le deep learning n'est pas une architecture unique mais une famille de modèles adaptés à chaque type de données. Le choix de l'architecture est guidé par l'inductif bias : les hypothèses implicites que le modèle fait sur la structure des données. Un bon inductif bias permet d'apprendre plus efficacement avec moins de données et de paramètres.

Le même algorithme d'optimisation (descente de gradient + retropropagation) unifie le paradigme du deep learning malgré la diversité des architectures. C'est cette unification qui fait la puissance et la généralité du deep learning moderne : MLP pour le tabulaire, CNN pour les images, RNN/LSTM pour les séquences, Transformers pour tout le reste.